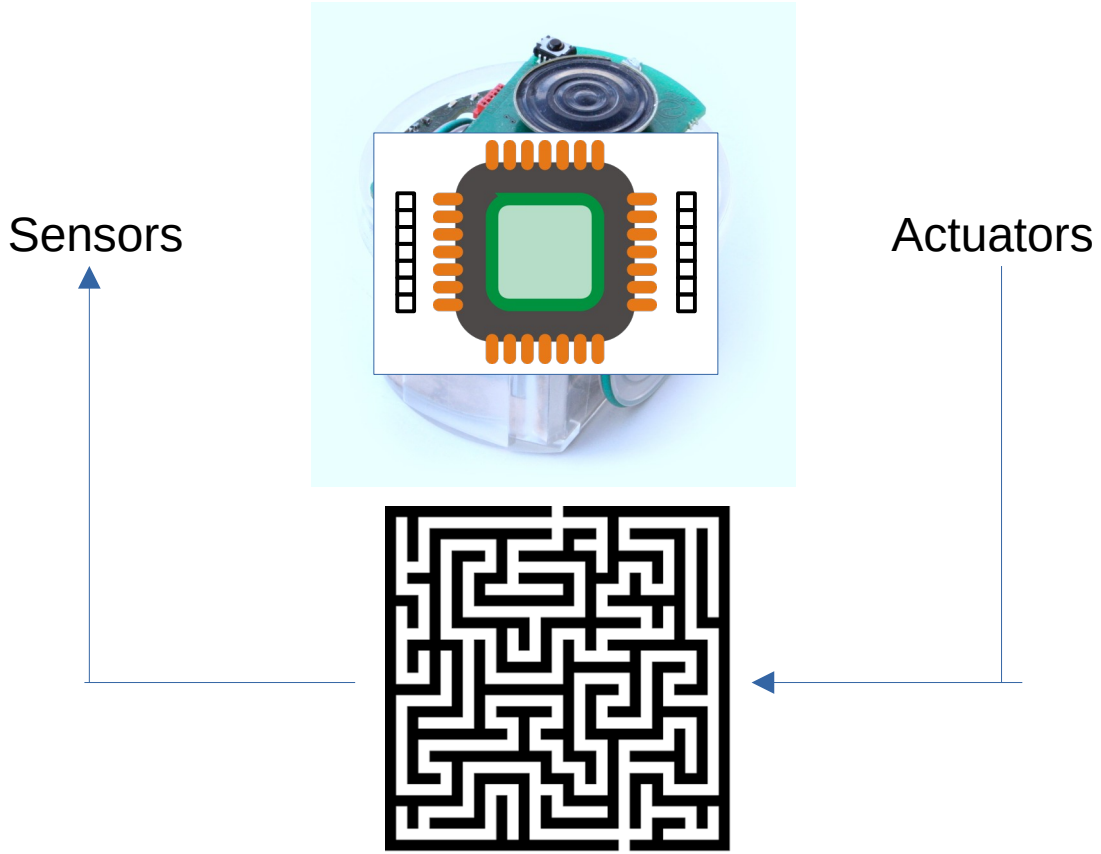
Artificial Intelligence for robotics

Part 7 : Neural network for robot control

Simon Gay

Artificial Intelligence for Robotics

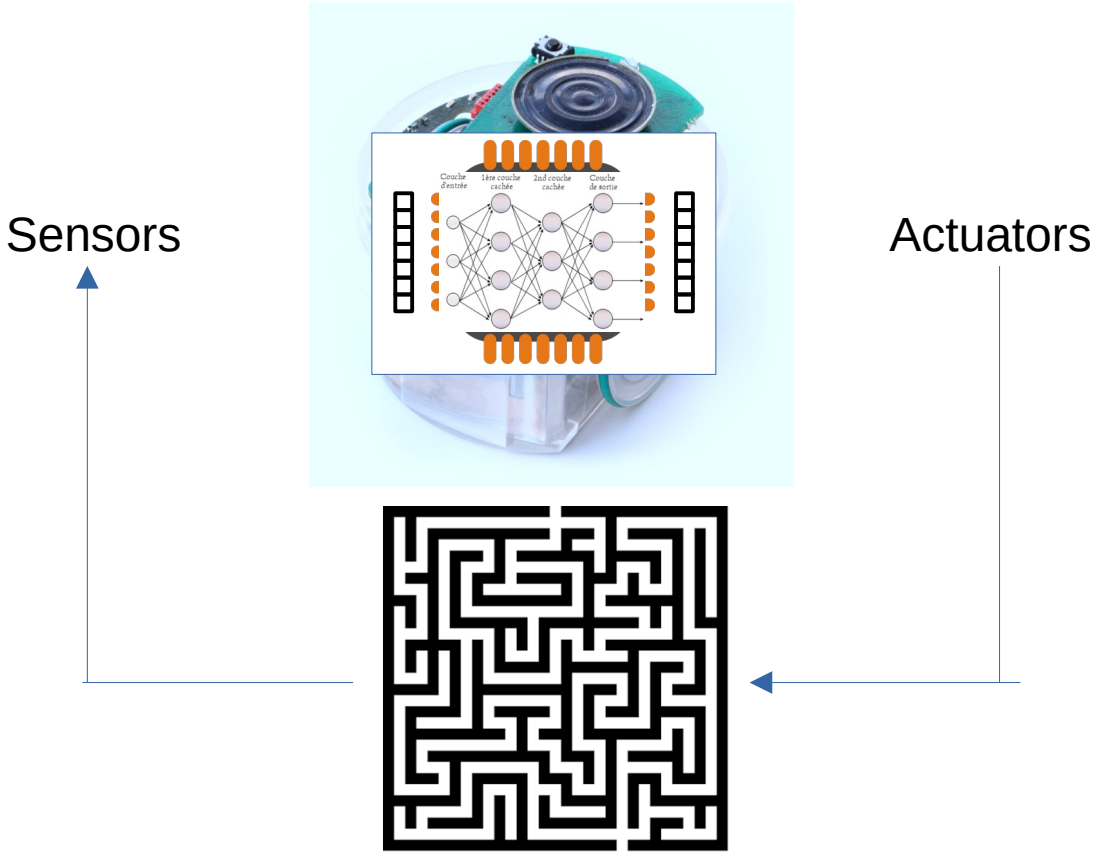
- **A robot is a system embodied in an environment, probing the environment with a set of sensors and act using actuators**
- From the controller's point of view, the sensor looks like a vector of value, and the controller pilots actuators by writing a vector of values
- The controller must define the output vector for each input vector



Artificial Intelligence for Robotics

- **A neural network can be defined between inputs and outputs**

- A large dataset is required to train the network on a large variety of case
  - The network is expected to react more or less relevantly when facing unknown input

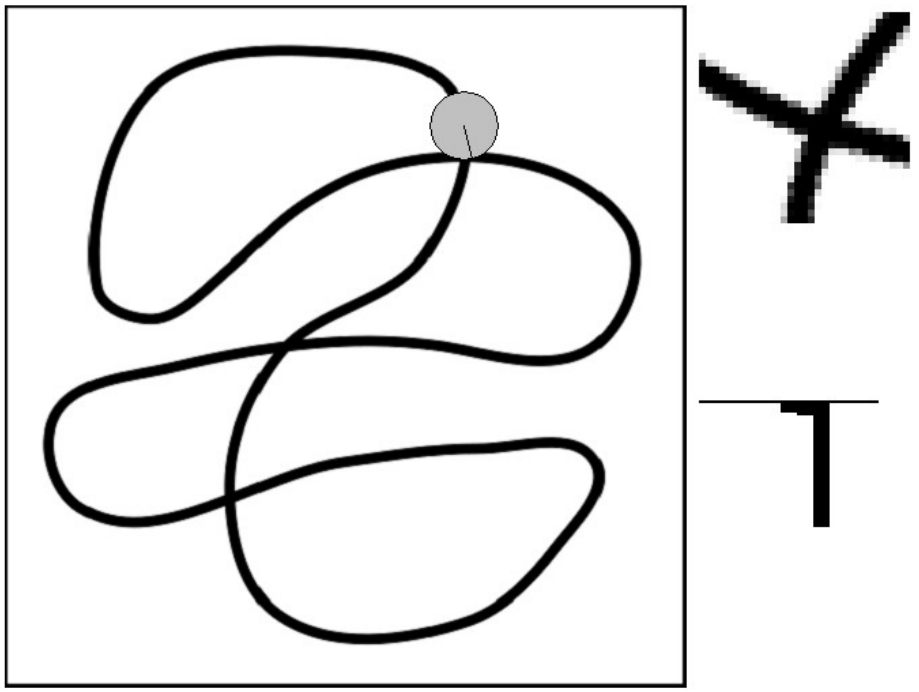


Artificial Intelligence for Robotics

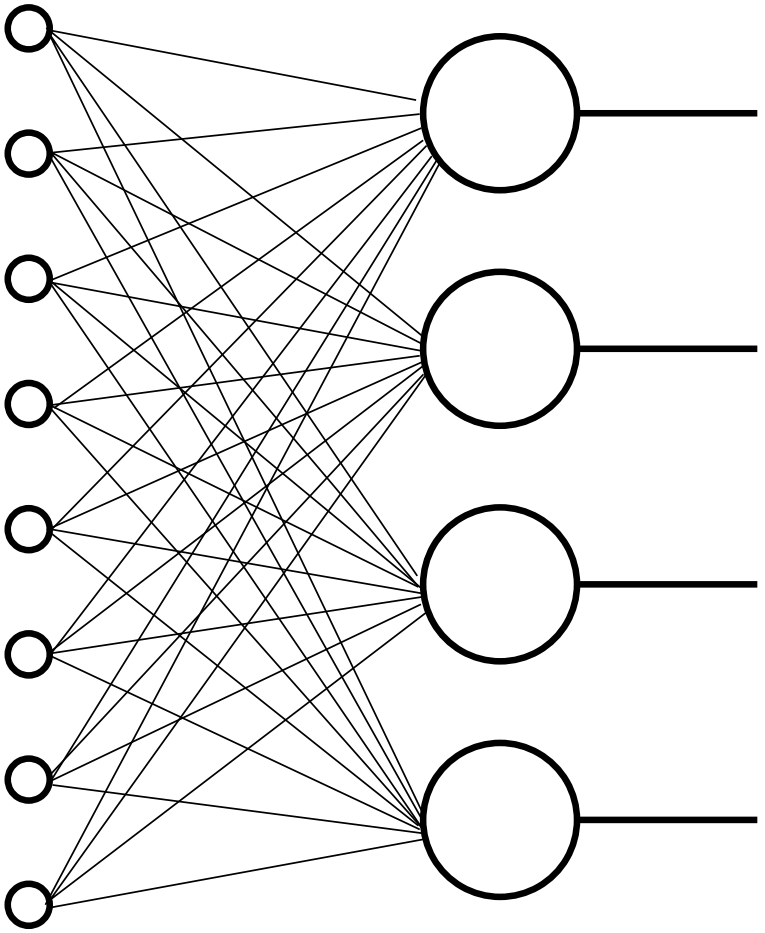
- **Our test environment : line follower robot**

- Sensor: camera of 31x31 pixels pointing toward the ground in front of the robot
- Actuators: differential propulsion, the controller provides the orientation

- Training offline then test with the simulated robot



Artificial Intelligence for Robotics



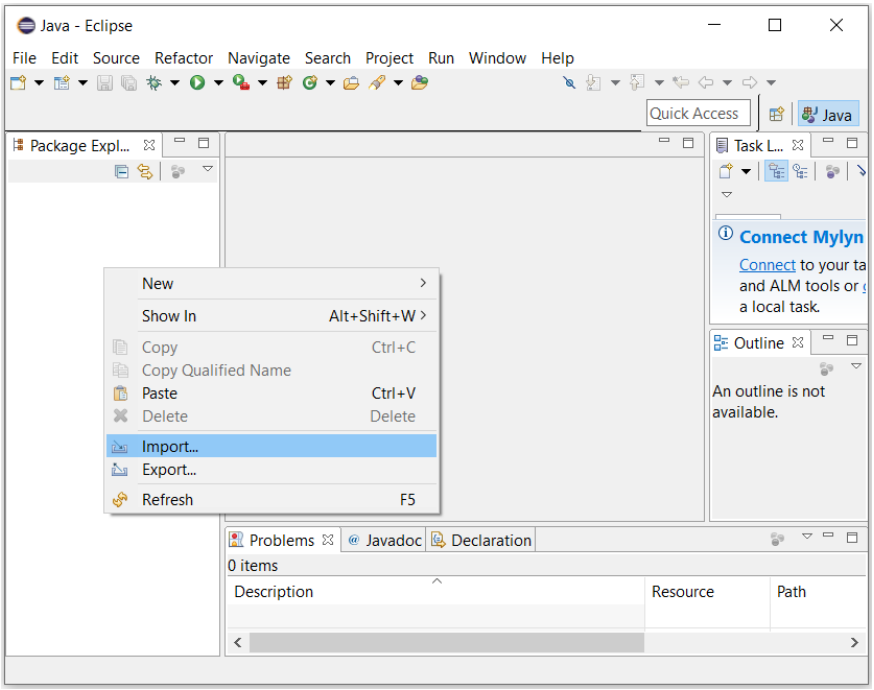
Let’s code !

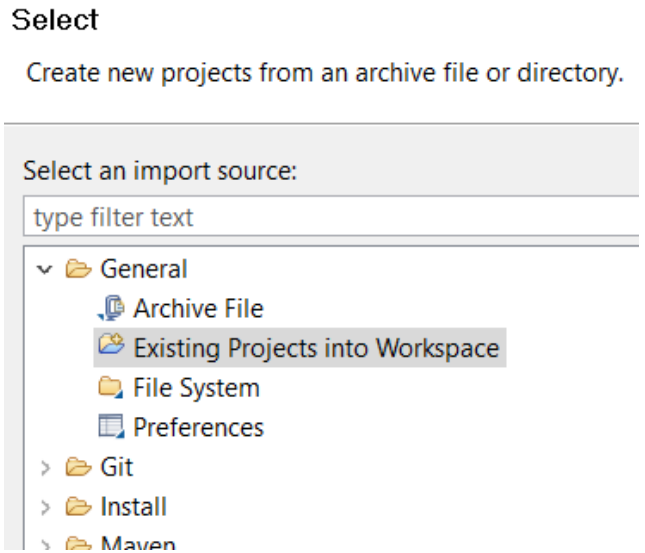
Artificial Intelligence for Robotics: the formal neuron

- **Environment**
  - Download on ***https://github.com/gaysimon/AI_for_robotics*** the project archive **‘4_line_follower.zip’,** image file **line1.png** and dataset file **‘dataset.txt’**
    - Do not unzip the project archive
    - Move the dataset file and image in a folder dedicated to this project
  - Open Eclipse and use your usual workspace folder

Artificial Intelligence for Robotics: the formal neuron

- **Environment**
  - In the Package Explorer, right-click in blank area and select “Import”
  - Select “General” then “Existing Projects into Workspace”, then click “Next”
  - Check “Select archive file”, select the archive “4_line_follower.zip” on your personal space, check the projects “4_line_follower_train” and “4_line_follower_exploit” then click on “Finish”





Artificial Intelligence for Robotics: the formal neuron

- **Environment**
  - The project “4_line_follower_train” contains 4 classes:
    - **Main**: contains the main function, perform the training process
    - **Layer**: layer of formal Neuron
    - **DisplayFrame** and **DisplayPanel**: display panel
  - The project “4_line_follower_exploit” contains 5 classes:
    - **Main**: contains the main function and control simulation loop
    - **Robot**: uses the neural network, contains functions used for simulation
    - **Layer**: layer of formal Neuron
    - **DisplayFrame** and **DisplayPanel**: display panel

Artificial Intelligence for Robotics: the formal neuron

- **We start with the training process: 4_line_follower_train**
  - First, import your neural network ‘library’: copy the content of your Layer class from project 2_single_layerV2 in the Layer class of project 4_line_follower_train
  - In class Main, set the SAVEPATH constant to the path to the directory containing the dataset→ this directory will also be used to save your trained network.
  - The images are 31x31 pixels: set two variables size_x and size_y to 31.
  - The robot’s orientation will be discretized into 11 directions (0→left, 5→ forward, 10→ right): set a variable nb_values to 11.

Artificial Intelligence for Robotics: the formal neuron

- **The training process**
  - To train the network, we will use the provided dataset that contains pixel values of a set of images and the orientation that must be applied. Each line correspond to the 31x31 pixel values, plus the expected output.
  - The function setData( ) loads this dataset as a 2D matrix ‘matrixImages’ containing the images and a 1D vector ‘matrixLabels’ containing the expected values. The function maps the orientation on the 0-10 interval.
  - Create a network composed of a unique layer

```
//-----------------
private DisplayFrame display;       // display panel

//-----------------
public Layer layer;             // Neuron Layer

//////////////
public Main(){

    // initialize structures
    layer=new Layer(size_x*size_y, nb_values);

    // load test matrix
    setData("dataset.txt");

    // initialize display frame
    display=new DisplayFrame(this);
```

Artificial Intelligence for Robotics: the formal neuron

- **The training process**
  - As the dataset is quite small, we will perform 500 epochs. Write the training process, similarly to the one in project 2.
  - The last part of the code saves the network's values as a text file (in PATH). Uncomment this part. Also uncomment the display part in DisplayPanel.

```
// initialize display frame
display=new DisplayFrame(this);

// epoch loop
for (epoch=0;epoch<500;epoch++){

    float sumdelta=0; // initialize the sum of errors

    // dataset loop
    for (test=0;test<matrixImages.length;test++){

        // define expected result
        int[] output=new int[nb_values];// vector of 0
        output[matrixLabels[test]]=1;   // answer at 1

        // process neuron
        layer.compute(matrixImages[test]);
        layer.learn(matrixImages[test], output);

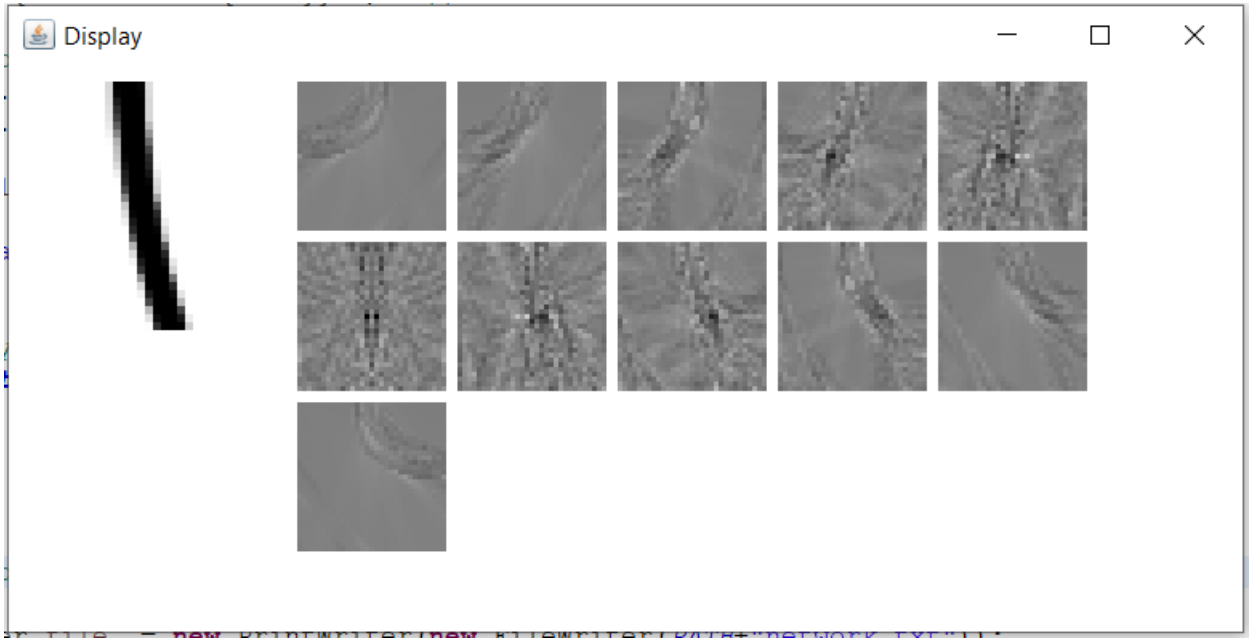
        sumdelta+=layer.sum_delta; // add error

        display.repaint(); // update display panel
        //try {Thread.sleep(100);
        //} catch (InterruptedException e) {e.printStackTrace();}
    }

    // display sum of errors
    System.out.println("epoch n°"+epoch+" : "+(sumdelta/matrixImages.length));
}
test=0; // avoid display's "out of range" error
```

Artificial Intelligence for Robotics: the formal neuron

- **The training process**
  - After training, you should get a network file ‘network.txt’
  - The neurons’ weights show the observation associated with each direction



Artificial Intelligence for Robotics: the formal neuron

- **Testing the network**
  - Let’s switch to the exploitation project
  - The Main class loads and sets the simulated environment, then starts the simulation loop.
  - The Robot class performs a simulation loop as follow:
    - Get the “camera” image
    - Define the direction → TODO
    - Perform the movement in simulated environment
  - The position and orientation are given by variables px, py and rz. The camera image is stored in a matrix of 31x31 integers

Artificial Intelligence for Robotics: the formal neuron

- **Testing the network**
  - First, we must convert the camera image into a vector of floats:
    - In class Robot, declare the vector and initialize it in constructor:

```
// position of the robot
public float px=70;
public float py=320;
public double rz=90;

public int[][] vision;
public float[] vector;
```

```
public Robot(Main m) {

    vision=new int[31][31];
    vector=new float[31*31];
```

    - Then, convert the image matrix into a vector in function ‘act( )’:

```
public void act() {

    generateImage();

    int nb=0;
    for (int i=0;i<31;i++){
        for (int j=0;j<31;j++){
            vector[nb]=(float)vision[j][i]/255f;
            nb++;
        }
    }
```

Artificial Intelligence for Robotics: the formal neuron

- **Testing the network**
  - Then create the neural network:
    - Copy the layer class from previous project
    - Declare the network and its parameters

```
public float[] vector;

public static int size_x=31;    // size of the image
public static int size_y=31;

public Layer layer;             // Neuron Layer
public static int nb_values=11;     // number of classes
```

    - Initialize the network and load weights with function ‘loadNetwork’ (uncomment it)

```
vector=new float[31*31];

// initialize structures
layer=new Layer(size_x*size_y, nb_values);
loadNetwork("network.txt");

display=new DisplayFrame(m);
```

Artificial Intelligence for Robotics: the formal neuron

- **Testing the network**
  - You can now use the network for robot's decision:
    - Compute the network on last vector image
    - Get neuron with strongest output (Max)
    - Note: the max id is then mapped to provide a direction. The robot perform 10 simulation sub-steps

```
public void act() {

    generateImage();

    int nb=0;
    for (int i=0;i<31;i++){
        for (int j=0;j<31;j++){
            vector[nb]=(float)vision[j][i]/255f;
            nb++;
        }
    }

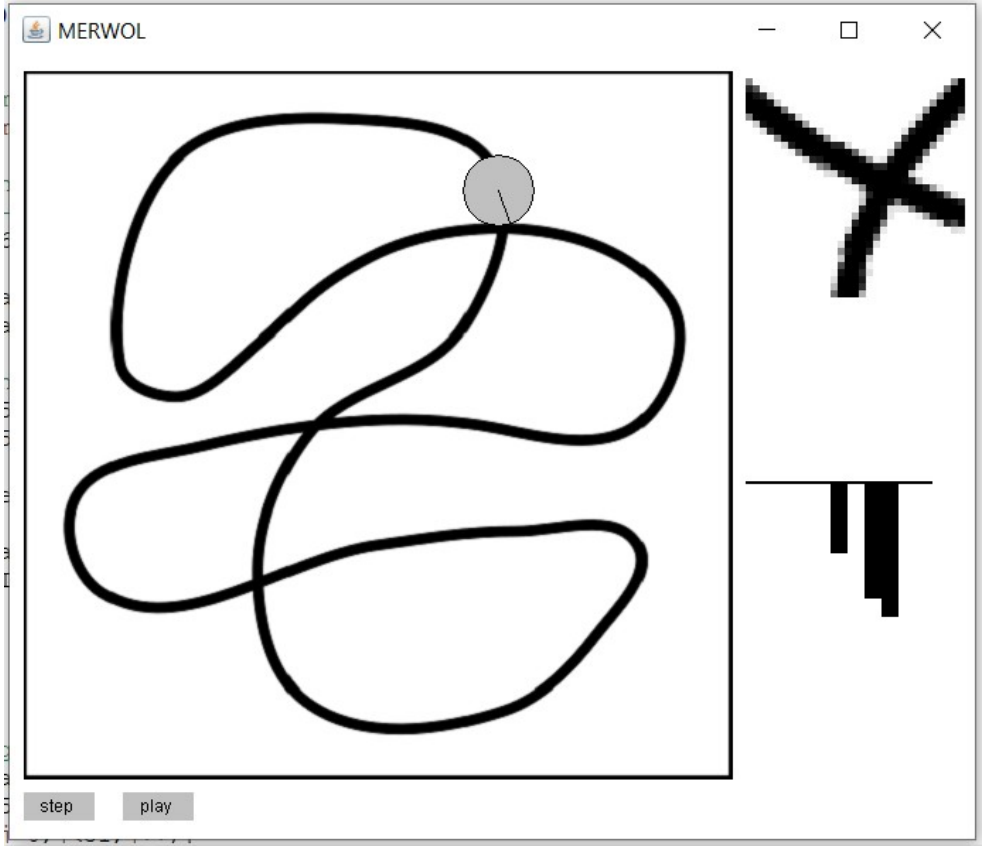
    layer.compute(vector);

    int imax=0;
    float max=0;
    for (int i=0;i<layer.results.length;i++){
        if (layer.results[i]>max){
            imax=i;
            max=layer.results[i];
        }
    }

    System.out.println(imax);
```

Artificial Intelligence for Robotics: the formal neuron

- **Testing the network**
  - The robot is now functional: start the simulation and click on ‘start’
    - You can observe neuron’s activity according to input image. Uncomment the displayPanel part showing neuron’s activity



Artificial Intelligence for Robotics: the formal neuron

- **What about using multi-layer network ? Will it be more efficient ?**
  - Replace the Layer from single layer project by Layer from multi-layerV2 project, and replace the robot's network for a 2-layers network (20-11)

```
//-----------------
public Layer[] layers;              // Neuron Layer

//////////////
public Main(){

    // initialize structures
    layers=new Layer[2];
    layers[0]=new Layer(size_x*size_y, nb_layer1);
    layers[1]=new Layer(nb_layer1, nb_layer2);



   // define expected result
   int[] output=new int[nb_layer2];// vector of 0
   output[matrixLabels[test]]=1;   // answer at 1

   //Forward propagation
   layers[0].compute(matrixImages[test]);
   layers[1].compute(layers[0].results);

   // backward propagation (deltas)
   layers[1].setLastDeltas(output);
   layers[0].setDeltas(layers[1]);

   //weights update
   layers[0].learn(matrixImages[test]);
   layers[1].learn(layers[0].results);

   sumdelta+=layers[1].sum_delta; // add error
```

```
public Layer[] layers;              // Neuron Layer
public static int nb_layer1=20;
public static int nb_layer2=11;     // number of classes


   // initialize structures
   layers=new Layer[2];
   layers[0]=new Layer(size_x*size_y, nb_layer1);
   layers[1]=new Layer(nb_layer1, nb_layer2);
   loadNetwork("network_multi_20_11.txt");



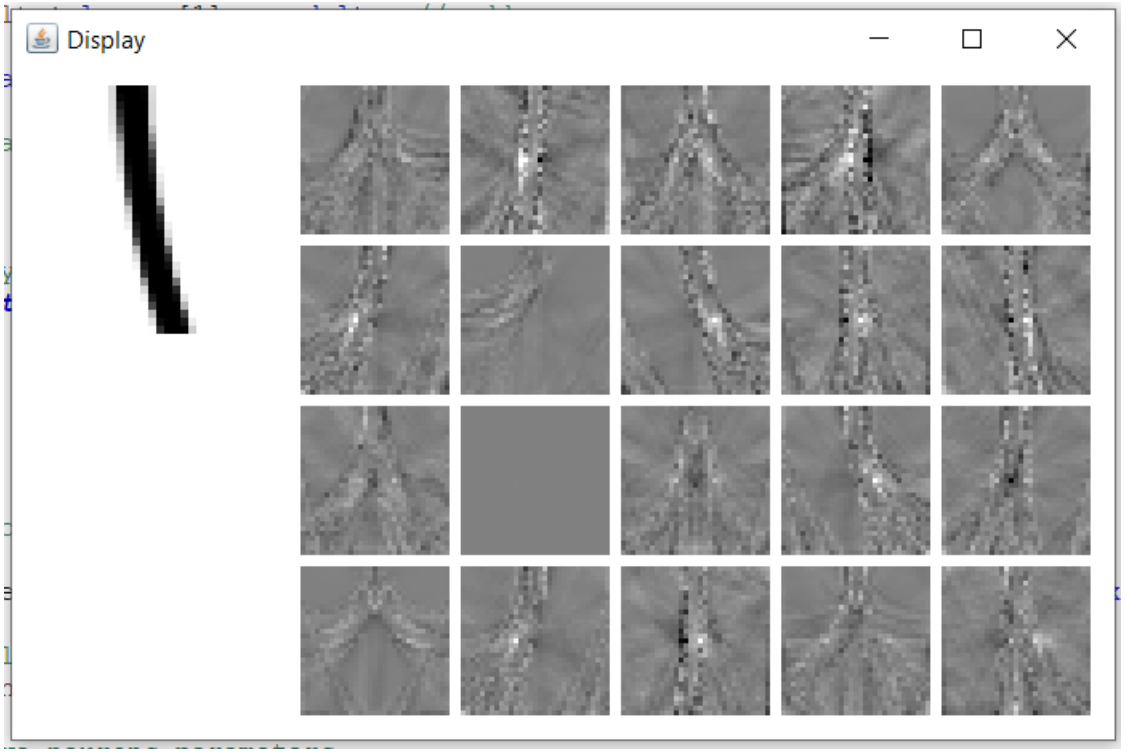
   layers[0].compute(vector);
   layers[1].compute(layers[0].results);

   int imax=0;
   float max=0;
   for (int i=0;i<layers[1].results.length;i++){
       if (layers[1].results[i]>max){
           imax=i;
           max=layers[1].results[i];
       }
   }

   System.out.println(imax);
```

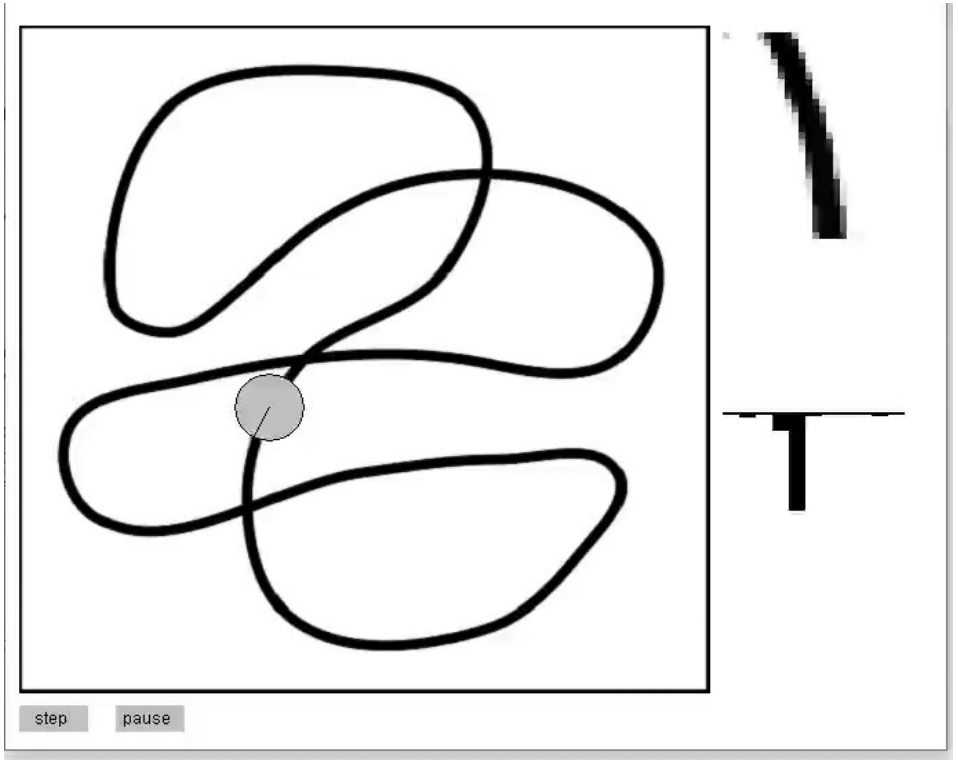
Artificial Intelligence for Robotics: the formal neuron

- **What about using multi-layer network ? Will it be more efficient ?**
  - After training, we can observe that some neurons of the first layer are not exploited



Artificial Intelligence for Robotics: the formal neuron

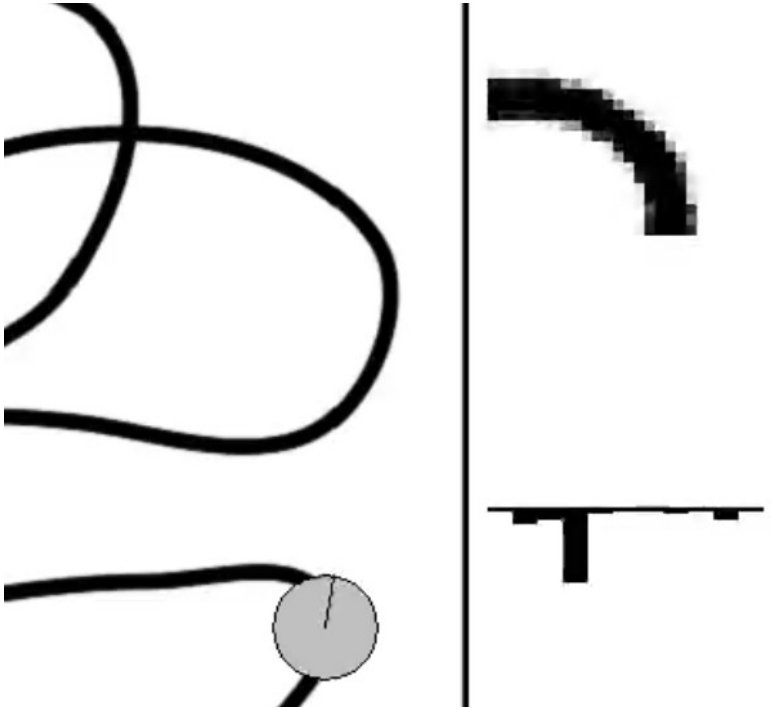
- **What about using multi-layer network ? Will it be more efficient ?**
  - Test in simulated environment



- What happened ?

Artificial Intelligence for Robotics: the formal neuron

- **What about using multi-layer network ? Will it be more efficient ?**
  - It seems that extreme values (0 and 10) do not react !
  - The reason is simple: the dataset is unbalanced as strong turns are rare
    - In single layer networks, neurons are independents, thus values 0 and 10 will learn even if the number of examples is low
    - On multi layer networks, first layer is trained to reduce the global error, thus features are defined to increase the success of most represented cases



- For multi-layer networks: the dataset must be, not only important in size, but also well balanced between classes. These networks are very efficient to detect data bias